

Technical Report

Optimizing LLM Routing: A Two-Stage Cost-Aware Architecture using SFT and GRPO

Router Team

May 30, 2026

Abstract

The rapid advancement of Large Language Models (LLMs) has introduced unprecedented capabilities, but deploying frontier models at scale incurs prohibitive inference costs. Model routing—dynamically directing user queries to either capable but expensive models or weaker but cheaper alternatives—has emerged as a critical solution [13]. However, traditional routing strategies (e.g., KNN, Emb+MLP) often struggle to balance this trade-off, either sacrificing too much accuracy or failing to meaningfully reduce costs.

In this technical report, we propose a novel Two-Stage Cost-Aware Routing Architecture that strictly balances hit rate and cost efficiency. We first utilize Supervised Fine-Tuning (SFT) to establish a stable routing cognitive baseline, followed by Group Relative Policy Optimization (GRPO) [17] to push the model toward its theoretical upper bound of cost-awareness. We evaluate our approach across multiple datasets (Hellaswag, MMLU, DISC) under varying LLM pool sizes. To effectively quantify success, we introduce two rigorous metrics: the Trade-off Efficiency Ratio (TER) and the accuracy-prioritized Net Benefit Score (NBS). Experimental results explicitly demonstrate that our **sft+grpo** variants consistently outperform traditional baselines, establishing a "Dominant" performance profile that secures near-peak accuracy while achieving precise, aggressive cost reductions.

Contents

1	Introduction	1
2	Background & Related Work	2
3	Methodology: Two-Stage Cost-Aware Router	2
3.1	Why a Two-Stage (SFT + GRPO) Framework	2
3.2	Post-SFT GRPO (RL Alignment) Specifications	2
4	Experimental Setup	5
4.1	Hardware & Environment	5
4.2	Task Formulation & Datasets	5
4.3	Evaluation Metrics	7
4.4	Models & Hyperparameter Configurations	8
5	Experimental Results: Efficiency and Cost Trade-off Comparisons	10
5.1	Performance Across Datasets	10
5.2	Training & Computation Time Efficiency	13
5.3	Inference Latency in Production (Batch Size = 1)	14
6	Cost-Aware Analysis	15
6.1	Alpha Penalty Spectrum	15
6.2	Routing Distribution Evolution: SFT to Cost-Aware GRPO	17
7	Root-Cause Analysis of Dataset Routing Value	20
7.1	The Four-Dimensional Analytical Framework	20
7.2	Empirical Base Data: LLM Win Rates	22
7.3	Two-Sided Origin Analysis: Where Does the Signal Come From	22
7.4	Visualizing the Dynamics: Embedding Spaces and Routing Heatmaps	24
8	Conclusion	28
A	Appendix	31
A.1	Prompt Architecture	31
A.2	System Prompt Template	31

1 Introduction

As the capabilities of Large Language Models (LLMs) continue to expand, so does the financial and computational burden of deploying them in production environments. Frontier models (e.g., GPT-4 class or 70B+ parameter models) deliver exceptional accuracy but are incredibly expensive to query. Conversely, smaller open-source models are cost-effective and fast but lack the complex reasoning capabilities required for difficult prompts.

To solve this dilemma, the industry is increasingly turning to **LLM Routing** [13]. A router is a lightweight mechanism that evaluates an incoming user query and determines the optimal model to handle it. The objective is to route simple queries to cheap models and reserve expensive models exclusively for difficult tasks.

However, achieving an optimal trade-off is mathematically and practically challenging. If a router is too aggressive in cutting costs, it sends difficult queries to weak models, heavily degrading system accuracy. If it is too conservative, it defaults to the expensive model, rendering the routing system useless. This report details the design, implementation, and rigorous experimental validation of a new routing architecture built on a two-stage SFT and GRPO [17] pipeline, aiming to solve the accuracy-cost paradox once and for all.

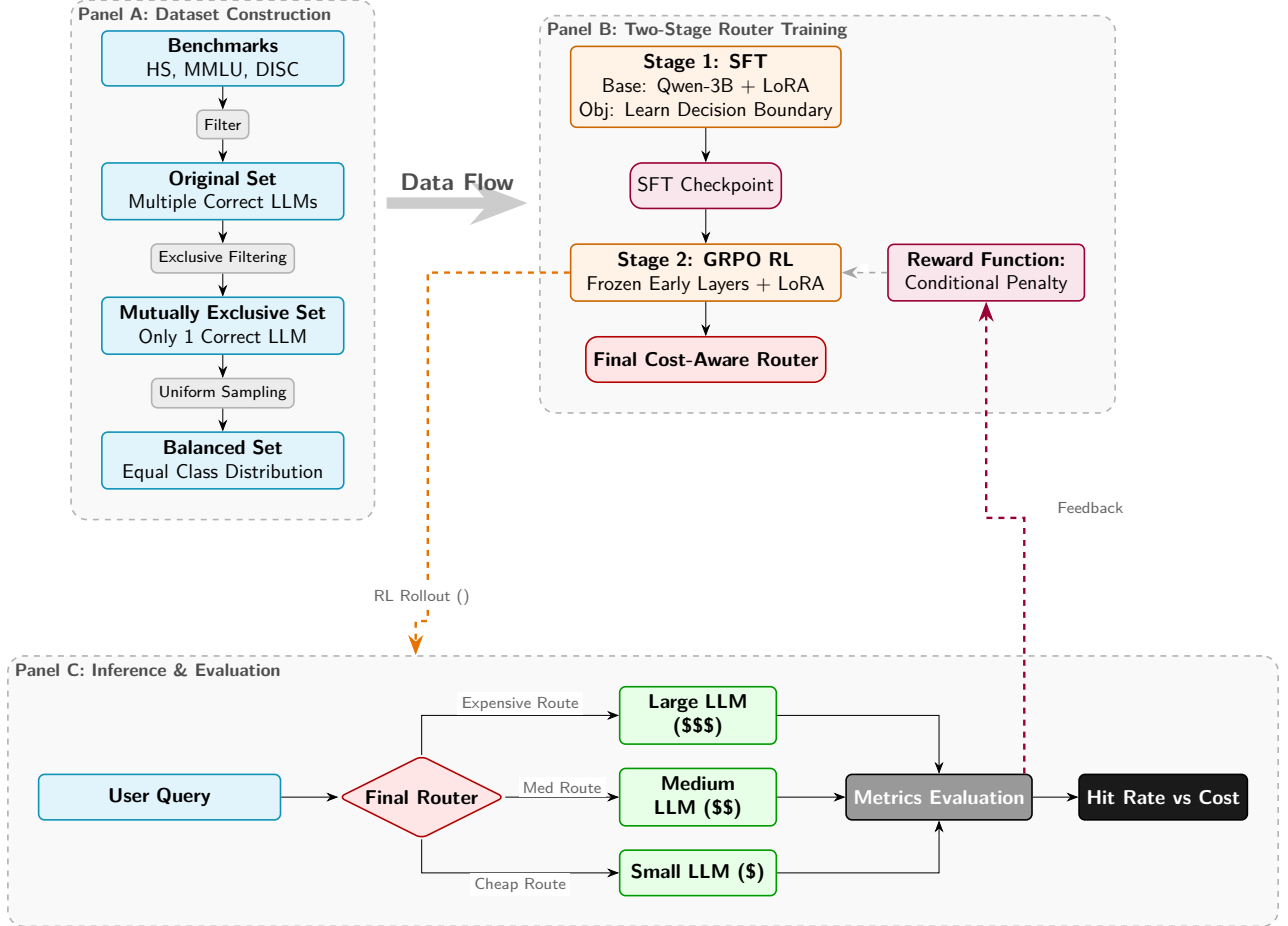


Figure 1: System Architecture Flowchart illustrating the end-to-end pipeline: (A) Dataset generation and filtration, (B) The proposed Two-Stage SFT and GRPO training framework, and (C) The final cost-aware inference evaluation. **Note:** The dashed colored lines represent the active dynamic feedback loop generated during the Stage 2 GRPO training phase.

2 Background & Related Work

Current approaches to LLM routing generally fall into two categories, both of which exhibit notable limitations:

- **Traditional ML Routers (KNN, Emb+MLP, BERT):** These methods typically rely on text embeddings [15, 20] mapped to historical performance data [9]. While easy to implement, they lack a deep semantic understanding of the tasks. As our experiments will show, they often create "lose-lose" scenarios where accuracy drops unacceptably relative to the money saved.
- **Pure Reinforcement Learning (RL) Routers:** Recent attempts have tried training routers entirely from scratch using RL to directly optimize for cost and accuracy rewards [24]. However, pure RL is notoriously unstable. Searching a vast action space without a solid baseline leads to slow convergence, extreme reward hacking, and high failure rates during training.

Recognizing these flaws, we hypothesize that combining the stability of supervised learning with the upper-bound exploration of RL is the most effective path forward.

3 Methodology: Two-Stage Cost-Aware Router

3.1 Why a Two-Stage (SFT + GRPO) Framework

Training a routing model entirely from scratch using purely Reinforcement Learning (such as GRPO [17]) is notoriously difficult. Pure RL approaches often suffer from extreme training instability, slow convergence, and struggle to effectively learn the underlying dataset distributions. To overcome these limitations, we employ a Two-Stage pipeline:

- **Stage 1 (SFT):** Provides a stable preliminary understanding of the dataset. By imitating optimal routing choices (Oracle labels), the model quickly establishes a robust baseline routing capability and aligns with the fundamental task format.
- **Stage 2 (GRPO):** Builds upon this stabilized foundation. Instead of searching blindly, the RL agent safely explores cost-accuracy trade-offs starting from a competent baseline. This effectively pushes the model’s routing performance toward its theoretical upper bound, proving to be the most stable and reliable training architecture in our experiments.

3.2 Post-SFT GRPO (RL Alignment) Specifications

In the second stage, we load the SFT model into the RL framework. To prevent the RL agent from collapsing while effectively teaching it cost-awareness, we implemented the following core components:

- **Partial LoRA Freezing (Targeted Decision-Boundary Updates):** Modern Large Language Models, such as our 36-layer Qwen-3B backbone, exhibit a hierarchical feature extraction architecture. Early and middle layers primarily encode universal semantics, syntax, and the foundational logical reasoning established during the SFT phase. Conversely, the final layers act as the "decision head," projecting these deep representations into specific action spaces.

Empirical testing revealed that applying RL gradient updates across all layers (or even half the network) forces the model to rapidly overwrite its core comprehension weights to hack the cost penalty, leading to severe Catastrophic Forgetting of the SFT cognitive baseline. To bypass this, we manually freeze layers 0 to 32, activating LoRA [7] adapters strictly on the **last 3**

Transformer layers. This effectively isolates the GRPO exploration purely to the final decision threshold. By fixing the feature-extraction pipeline and only tuning the decision head, we confirm that these 3 layers provide more than enough parameter space to learn rigorous cost-awareness without degrading the structural knowledge acquired in Stage 1. **except the last 3 Transformer layers’ LoRA [7] adapters.** This strictly restricts RL exploration to the final decision-making boundaries.

- **Staged Utility Reward Function (Cost-Accuracy Derivation):** We designed a composite reward function that explicitly penalizes expensive routing decisions without destabilizing the training process. The derivation follows two distinct steps:

Step 1: Cost Quantification & Normalization. First, we define the true financial burden (**Real Cost**) of a candidate model based on its API pricing and average token consumption:

$$Cost_{real} = Avg_Tokens \times \frac{Prompt_Price + Completion_Price}{2}$$

Directly subtracting raw monetary values from an RL reward signal causes extreme gradient instability. Therefore, we project these costs into a normalized penalty space $P_{cost} \in (0, 1]$ using Max Normalization across the candidate LLM pool:

$$P_{cost} = \frac{Cost_{real}}{\max(Cost_{pool})}$$

This scaling ensures that the most expensive model in the pool always receives a maximum penalty severity of 1.0, while cheaper alternatives receive proportionally smaller penalties.

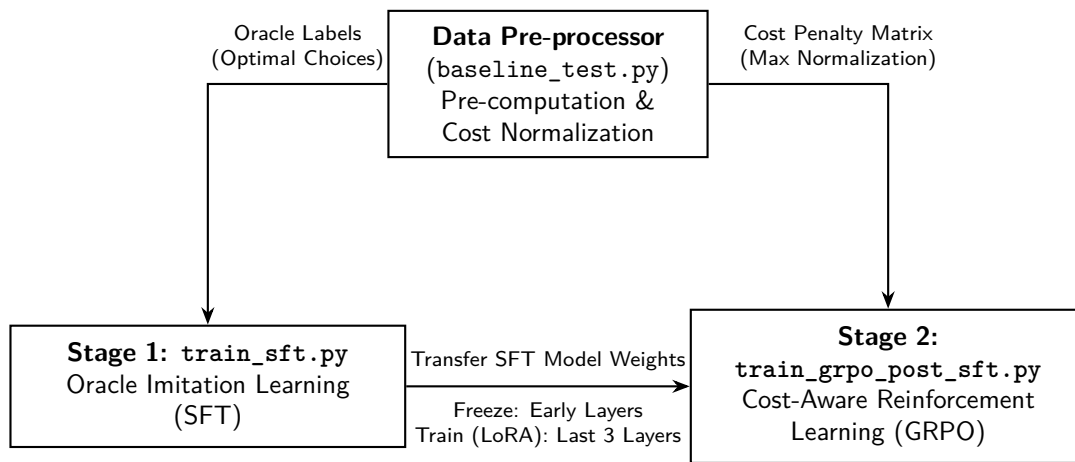
Step 2: Composite Reward Calculation. For each generated routing decision, the final reward incorporates this normalized penalty:

$$Reward = B_{format} + \begin{cases} \max(0, R_{base} - \alpha \cdot P_{cost}), & \text{if } Score > 0 \\ 0, & \text{if } Score = 0 \end{cases}$$

Where B_{format} (0.05) stabilizes early output format, R_{base} is the correctness baseline (1.0), and α is the hyperparameter strictly controlling the **Cost Penalty Weight**. By smoothly tuning α , we manipulate how aggressively the router avoids the maximum penalty, enforcing strict budget-awareness.

- **Custom Validation Checkpointing (val_stop):** A custom `TrainerCallback` suspends training every 50 steps, with ($Temperature = 0.1$), and evaluates a hold-out Validation Set. It autonomously saves the checkpoint yielding the highest reward, completely eliminating RL reward hacking.

Two-Stage Cost-Aware Router Training Pipeline



4 Experimental Setup

This section details the hardware environment, task formulation, dataset construction, hyperparameter configurations, and the core metrics used to evaluate the routing algorithms.

4.1 Hardware & Environment

All model training and inference experiments were conducted on a single NVIDIA GeForce RTX 3090 GPU (24GB VRAM). To ensure the smooth fine-tuning of Large Language Models within a constrained VRAM environment, we utilized 4-bit quantization techniques (`bnb-4bit`) [5] and explicitly capped the GPU memory utilization at `gpu_memory_utilization = 0.5`.

This specific 50% allocation (approximately 12 GB) was highly intentional. During the GRPO Reinforcement Learning phase, the auto-regressive generation (rollout) requires massive dynamic memory. Capping the model allocation strictly reserves the majority of the remaining VRAM for the expansive KV cache. Furthermore, empirical testing confirmed that the training loop is generation-bound rather than purely compute-bound; increasing the utilization limit to 0.9 yielded identical training times, making 0.5 the optimal configuration for stability and efficiency.

4.2 Task Formulation & Datasets

The primary objective of the Router is to accurately predict which LLM possesses the capability to answer a given user query. In scenarios where multiple models can answer correctly, the Router must learn to preferentially select the most cost-effective option. To construct a comprehensive evaluation and training environment, we sourced data from highly established benchmarks and processed them into specific training configurations.

4.2.1 Source Benchmarks

Our experiments leverage three distinct datasets, each challenging the routing architecture from a different cognitive dimension:

- **HellaSwag (Adversarial Commonsense Reasoning)** [23]: Developed by researchers at the University of Washington and AI2, this benchmark is designed to evaluate a model’s grasp of real-world physical and logical consistency rather than static factual knowledge. It uniquely employs “Adversarial Filtering” to strip away easily guessable distractors. The remaining options are trivial for humans ($\sim 95\%$ accuracy) but highly deceptive for models relying purely on statistical patterns. **Routing Value:** HellaSwag provides the strongest “logic signal” in our experiments, forcing the router to identify which candidate models possess genuine commonsense consistency to avoid adversarial traps.
- **MMLU (Massive Multitask Language Understanding)** [6]: Proposed by UC Berkeley, MMLU is widely recognized as the premier benchmark for evaluating the breadth and depth of a model’s static knowledge acquired during pre-training. It spans 57 diverse subjects, ranging from elementary mathematics to advanced professional exams in law, ethics, and medicine. **Routing Value:** MMLU exhibits extreme “feature sparsity” due to its vast domain coverage. It serves as the ultimate test for the router’s “specialized dispatch” capability—requiring it to accurately determine, for instance, when an expensive expert model is strictly necessary to resolve a highly specialized medical query.
- **DISC (Discovery Set - Production-Oriented Hybrid):** DISC is a proprietary hybrid dataset engineered specifically for this project. It merges the foundational elements of HellaSwag

(reasoning) and MMLU (knowledge) while deeply integrating the core data from the **Berkeley Function Calling Benchmark V2 (BFCL V2)** [14]. BFCL V2 focuses heavily on "Tool-Use" scenarios, utilizing real-world API documentation and user queries to evaluate multi-objective decision-making, parallel invocation, and relevance detection (i.e., knowing when *not* to use a tool). **Routing Value & Strategic Importance:** DISC is the most production-ready environment in our study. By layering BFCL V2's "action accuracy" over the background noise of logic (HS) and breadth (MMLU), it forces the router to precisely classify whether an incoming query demands deep reasoning, broad retrieval, or complex API execution. This perfectly mirrors the core challenges faced by real-world LLM Gateways.

4.2.2 Data Partitions

To incrementally build the router's capability using these source benchmarks, we categorized our training data into three distinct tiers:

- **Original Set:** An unfiltered, real-world dataset distribution. In this set, a simple query might be answered correctly by multiple LLMs simultaneously ($Score = 1$). This dataset is primarily utilized during the Reinforcement Learning (GRPO) stage to perform cost-aware fine-tuning, teaching the Router to identify the highest ROI (Return on Investment) model among multiple valid solutions.
- **Mutually Exclusive Set:** Data strictly filtered from the Original Set so that *only one* specific model answers correctly. This extreme data characteristic provides highly distinct decision boundaries. We deploy this dataset exclusively during Supervised Fine-Tuning (SFT) to establish a highly accurate and stable foundational routing baseline.
- **Balanced Set (Mutually Exclusive Balanced):** In the Mutually Exclusive Set, powerful (and expensive) models typically dominate the exclusive correct answers, leading to severe data imbalance. To counter this, we applied uniform sampling over the Mutually Exclusive Set, forcing an equal number of training instances for each candidate model, thereby eliminating representation bias during GRPO.

4.2.3 Data Distribution Integrity in Phase-1 SFT

When building a two-stage routing architecture, an intuitive but fatal pitfall is applying "Artificial Balancing" or "Extreme Cost-Filtering" to the training data during the SFT phase. Empirical evidence shows that destroying the Ground-Truth (GT) distribution leads to severe accuracy degradation and blurred decision boundaries during inference.

- **The Pitfall of Artificial Balancing:** The core objective of the SFT phase is to establish foundational cognitive boundaries for expert routing. This relies heavily on the model learning both feature weights and real-world Prior Probabilities. Suppose Model A has a 40% win rate in the true distribution, while a lightweight Model B specializes in a niche domain with a 4% win rate. If we use uniform sampling to artificially balance their training samples, the router develops a severe cognitive bias. It over-amplifies Model B's feature weights and loses its grasp on Model A's global capabilities, leading to frequent False Positives where general queries are mistakenly routed to Model B.
- **The Danger of Extreme Cost-Filtering:** Another common error is strictly filtering labels in the SFT phase to only keep the "cheapest correct model." While seemingly cost-effective, this deprives the router of learning the "Capability Ceiling" of expensive frontier models. If a

powerful model only appears as the target for extremely difficult prompts, the router fails to build a complete map of its expertise, leading to fatal misroutes when cheap models fall short.

Conclusion: Therefore, during the foundational SFT phase, a non-negotiable principle is that the combination ratio of Ground-Truth labels must undergo **Proportional Scaling** that preserves the original true distribution. Maintaining this integrity ensures the router accurately anchors the capability boundaries of each LLM, providing a solid decision baseline for the subsequent GRPO cost-aware training.

4.2.4 Candidate LLM Pool

To rigorously evaluate the routing architecture’s ability to balance cost and accuracy, we established a diverse candidate pool of Large Language Models. This pool intentionally encompasses a wide spectrum of capabilities, ranging from expensive, high-parameter frontier models to highly cost-effective, specialized small language models (SLMs). The full pool utilized across our datasets (notably the 6-LLM environments) includes:

- **Llama-4-Maverick-17B & Llama-4-Scout-17B** [10, 11]: Serving as our top-tier and mid-tier reasoning baselines, these models represent the frontier of open-weight capabilities, offering exceptional logic resolution at a premium token cost.
- **gpt-oss-20b** [2]: A highly capable 20-billion parameter model that frequently serves as a stable bridge between heavy reasoning tasks and standard text generation, offering a strong balance of accuracy and efficiency.
- **Mistral-Small-3.1** [12]: Selected for its extreme efficiency and high baseline performance in specific knowledge retrieval domains and fast-inference scenarios.
- **Google-Gemma-3-27B** [18]: Integrated to provide a robust, mid-weight alternative with distinct training data distributions, which is crucial for avoiding homogenous capability overlaps in the routing pool.
- **Microsoft-Phi-4** [1]: Representing the ultra-efficient SLM tier. While possessing the lowest cost, it demands high routing precision to be utilized effectively without degrading overall system accuracy.

The true cost ($Cost_{real}$) of each model used during the GRPO reward calculation is strictly derived from their respective public API pricing brackets combined with the average empirical token consumption per query.

4.3 Evaluation Metrics

In our routing framework, the response quality of each candidate LLM is evaluated using binary labels (Correct = 1, Incorrect = 0). The comprehensive performance of the Router is quantified using four core metrics:

1. Hit Rate (HR) ($\uparrow$): The primary metric for measuring router accuracy. A routing decision is considered a "Hit" if the selected LLM’s score matches the maximum score achievable among all available candidate models for that specific query.

$$\text{Hit} \iff Score_{selected} = \max(Score_{all_models})$$

2. Cost ($\downarrow$): The financial cost incurred by the routing decision. It is defined as the product of the selected model’s average token usage and its specific API pricing structure.

$$\text{Cost} = \text{Avg_Tokens} \times \frac{\text{Prompt_Price} + \text{Completion_Price}}{2}$$

3. TER (Trade-off Efficiency Ratio) ($\uparrow$):

$$TER = \frac{\text{Cost Savings (\%)}}{\text{Hit Rate Sacrifice (\%)}}$$

Represents the "exchange rate" of the routing trade-off (i.e., how much budget is saved for every 1% drop in accuracy). If a router simultaneously lowers accuracy and increases cost, the trade-off is marked as **Inv** (Invalid). A router that strictly improves or matches both metrics compared to the strongest baseline achieves a **Dominant** status.

4. NBS (Net Benefit Score) ($\uparrow$):

$$NBS = 3 \times \Delta\text{Hit Rate (\%)} + \Delta\text{Cost Savings (\%)}$$

An accuracy-prioritized metric designed to calculate the absolute net gain of the router.

Baseline Definition: The Δ (delta) values are strictly calculated relative to the performance of the **Strongest Baseline** model in the candidate pool for that specific dataset.

Weighting Justification (3:1 Ratio): In LLM routing, cost reductions scale much more aggressively and violently than accuracy drops. A naive 1:1 weighting allows a router to easily "hack" the metric by indiscriminately sending all queries to the cheapest, lowest-performing model, resulting in heavily inflated scores despite a catastrophic loss in quality. The 3:1 accuracy-to-cost ratio acts as a strict regulatory penalty, ensuring that cost savings are only rewarded if the baseline accuracy is rigorously protected. To provide a comprehensive evaluation, NBS is intended to be analyzed in tandem with TER (which evaluates the relative exchange rate).

4.4 Models & Hyperparameter Configurations

4.4.1 Core Routing Architecture (SFT + GRPO)

Our proposed two-stage methodology is built upon a robust LLM backbone and optimized via Low-Rank Adaptation (LoRA) [7]:

- **Router Backbone:** unsloth/Qwen2.5-3B-Instruct-bnb-4bit [22].
- **LoRA Configuration:** Applied in both stages with $\text{Rank}(r) = 16$ and $\text{Alpha}(\alpha) = 16$ [7].
- **Stage 1 (SFT):** Trained on the Balanced Set to build routing intuition. Learning Rate = 2×10^{-4} , Equivalent Batch Size = 8.
- **Stage 2 (Post-SFT GRPO):** Inherits SFT weights and trains on the Original Set. To firmly protect the SFT cognitive boundary, training is constrained to **650 steps** with an ultra-low Learning Rate (5×10^{-7}). We apply Weight Decay (0.01) and Warmup Ratio (0.05). A high KL Penalty ($\beta = 0.2$) is enforced to deeply anchor the model to its SFT behavior, while Generation Temperature is constrained to 0.9. Rollout = 8.

LoRA Freezing: To prevent catastrophic forgetting, the front network is frozen, and only the final 3 layers are updated during RL.

Penalty Mechanism: The Alpha cost penalty is fixed at 0.2 without warmup, utilizing a **Conditional Penalty**—excess cost is only penalized if the selected model is more expensive than the cheapest capable model, reinforcing precision.

4.4.2 Baseline Models

To validate our approach, we compared it against traditional Machine Learning routers and advanced baseline methods. These baselines convert text into vectors using embedding models before executing routing predictions:

- **Embedding Models:** By default, baselines utilize `sentence-transformers/all-MiniLM-L6-v2` [15, 20]. The **BERT-MLC** baseline utilizes the more powerful `answerdotai/ModernBERT-base` [21].
- **KNN:** K value set to 10 [4].
- **SW-Ranking** [13]: A robust routing baseline derived from the Chatbot Arena Strong-Win methodology. K value set to 10, Softmax Temperature parameter set to 0.8.
- **Matrix Factorization (MF)** [8, 13]: Latent Dimension = 64, Learning Rate = 0.001, Batch Size = 32, Training Epochs = 50.
- **BERT-MLC** [13]: Training Batch Size = 10, Evaluation Batch Size = 12.
- **Pure GRPO (R1-style [24] / `grpo-r1`):** A pure Reinforcement Learning baseline trained from scratch without SFT priors. It requires extensive exploration over **2500 steps** with a larger Learning Rate (1×10^{-5}), Weight Decay (0.1), and Warmup Ratio (0.1). To encourage widespread exploration, the KL Penalty is lowered ($\beta = 0.16$), Generation Temperature is high (1.0), and the entire LoRA network is unfrozen. It utilizes an **Alpha Warmup** strategy: the first 20% of steps ignore cost ($\alpha = 0$), gradually scaling to 0.15 to prevent early reward collapse.
- **Oracle / Random:** *Oracle* represents the theoretical upper limit (guaranteeing 100% selection of the highest-scoring model), while *Random* serves as the baseline lower limit.

5 Experimental Results: Efficiency and Cost Trade-off Comparisons

5.1 Performance Across Datasets

Table 1: Performance and Efficiency on Hellaswag ($|\text{LLM}| = 3$) Dataset ($\alpha = 0.2$)

Key Takeaway: Because the Strongest Baseline’s base cost is already extremely low (204.51), standard routers struggle. They create a "lose-lose" scenario, rendering TER Invalid and resulting in heavily negative NBS. However, our proposed sft+grpo methods break this barrier. They are the only strategies to achieve Dominant status, delivering both higher Hit Rates and lower costs, resulting in a solid positive NBS.

Strategy	Standard (Normal)				Exclusive (exclu)			
	HR ($\uparrow$)	Cost ($\downarrow$)	TER ($\uparrow$)	NBS ($\uparrow$)	HR ($\uparrow$)	Cost ($\downarrow$)	TER ($\uparrow$)	NBS ($\uparrow$)
Strongest Baseline	0.8969	204.51	-	-	0.8969	204.51	-	-
Largest Baseline	0.8969	204.51	-	-	0.8969	204.51	-	-
Smallest Baseline	0.6341	200.15	0.07	-85.76	0.6341	200.15	0.07	-85.76
Router (oracle)	1.0000	208.70	Inv	+32.44	1.0000	208.70	Inv	+32.44
Router (random)	0.7770	221.85	Inv	-48.58	0.7788	222.22	Inv	-48.16
Router (knn)	0.8924	208.59	Inv	-3.50	0.8957	206.10	Inv	-1.18
Router (emb+MLP)	0.8730	218.68	Inv	-14.92	0.8875	209.66	Inv	-5.66
Router (emb+KNN+ELO)	0.8893	211.03	Inv	-5.73	0.8912	207.88	Inv	-3.55
Router (BERT_MLC)	0.8899	207.74	Inv	-3.92	0.8794	214.88	Inv	-10.92
Router (Qwen2.5-3B Base)	0.7388	248.54	Inv	-68.96	-	-	-	-
Router (sft-oracle)	-	-	-	-	0.8710	215.83	Inv	-14.20
Router (grpo-rl)	0.8937	208.07	Inv	-2.81	0.8890	211.96	Inv	-6.29
Router (sft+grpo) exclu+exclu	-	-	-	-	0.9013	203.52	Dominant	+1.96
Router (sft+grpo) exclu+balance	-	-	-	-	0.9024	203.43	Dominant	+2.37
Router (sft+grpo) exclu+origin	-	-	-	-	0.8996	204.11	Dominant	+1.10

Table 2: Performance and Efficiency on Hellaswag ($|\text{LLM}| = 6$) Dataset ($\alpha = 0.2$)

Key Takeaway: With an expanded model pool ($|\text{LLM}| = 6$), traditional routers can find better cost-saving paths, occasionally yielding a positive NBS in Standard mode. However, in Exclusive mode, their accuracy often drops severely, dragging NBS back into the negative. Our *sft+grpo* strategies perfectly balance the trade-off, restricting Hit Rate sacrifice to just $\sim 1.1\%$ while maximizing cost efficiency, securing the highest TER and a robust positive NBS among non-oracle models.

Strategy	Standard (Normal)				Exclusive (exclu)			
	HR ($\uparrow$)	Cost ($\downarrow$)	TER ($\uparrow$)	NBS ($\uparrow$)	HR ($\uparrow$)	Cost ($\downarrow$)	TER ($\uparrow$)	NBS ($\uparrow$)
Strongest Baseline	0.8468	204.52	-	-	0.8468	204.52	-	-
Largest Baseline	0.8468	204.52	-	-	0.8468	204.52	-	-
Smallest Baseline	0.4406	40.95	1.67	-63.93	0.4406	40.95	1.67	-63.93
Router (oracle)	1.0000	103.66	Dominant	+103.58	1.0000	103.66	Dominant	+103.58
Router (random)	0.6478	141.05	1.32	-39.47	0.6572	143.28	1.34	-37.23
Router (knn)	0.8147	165.94	4.98	+7.49	0.7865	160.84	3.00	-0.01
Router (emb+MLP)	0.8060	154.98	5.03	+9.77	0.7703	155.48	2.65	-3.12
Router (emb+KNN+ELO)	0.8157	166.58	5.05	+7.53	0.7857	161.36	2.92	-0.54
Router (BERT_MLC)	0.7887	145.27	4.22	+8.39	0.7885	171.35	2.36	-4.43
Router (Qwen2.5-3B Base)	0.6462	133.49	1.73	-25.45	-	-	-	-
Router (sft-oracle)	-	-	-	-	0.7573	156.43	2.22	-8.19
Router (grpo-rl)	0.7550	135.50	3.67	+6.20	0.7320	158.20	1.97	-11.80
Router (sft+grpo) exclu+exclu	-	-	-	-	0.8373	189.45	6.57	+4.00
Router (sft+grpo) exclu+balance	-	-	-	-	0.8367	189.50	6.16	+3.77
Router (sft+grpo) exclu+origin	-	-	-	-	0.8243	176.48	5.16	+5.74

Table 3: Performance and Efficiency on MMLU ($|\text{LLM}| = 6$) Dataset ($\alpha = 0.2$)

Key Takeaway: MMLU exposes the defensive robustness of the proposed architecture. In a chaotic dataset with extreme feature overlap, traditional routers make blind guesses to cut costs, heavily degrading accuracy and crashing into deeply negative NBS. Instead of forcing a flawed compromise, *SFT+GRPO* recognizes the lack of clear decision boundaries and executes a **defensive fallback**—safely degenerating toward the strongest baseline to strictly preserve system accuracy. This fail-safe behavior highlights the architecture’s true engineering value in unpredictable production environments.

Strategy	Standard (Normal)				Exclusive (exclu)			
	HR ($\uparrow$)	Cost ($\downarrow$)	TER ($\uparrow$)	NBS ($\uparrow$)	HR ($\uparrow$)	Cost ($\downarrow$)	TER ($\uparrow$)	NBS ($\uparrow$)
Strongest Baseline	0.8971	284.56	-	-	0.8971	284.56	-	-
Largest Baseline	0.8971	284.56	-	-	0.8971	284.56	-	-
Smallest Baseline	0.7387	59.40	4.48	+26.14	0.7387	59.40	4.48	+26.14
Router (oracle)	1.0000	140.14	Dominant	+85.16	1.0000	140.14	Dominant	+85.16
Router (random)	0.7743	201.30	2.14	-11.81	0.7621	200.37	1.97	-15.56
Router (knn)	0.8582	239.58	3.65	+2.80	0.8281	212.55	3.29	+2.23
Router (emb+MLP)	0.8374	225.93	3.10	+0.64	0.8070	212.23	2.53	-4.71
Router (emb+KNN+ELO)	0.8582	241.79	3.47	+2.02	0.8304	214.22	3.33	+2.41
Router (BERT_MLC)	0.8005	125.58	5.19	+23.57	0.8340	199.31	4.26	+8.86
Router (Qwen2.5-3B Base)	0.6568	142.48	2.08	-22.16	-	-	-	-
Router (sft-oracle)	-	-	-	-	0.7977	218.89	2.08	-10.16
Router (grpo-rl)	0.7481	109.95	3.70	+11.53	0.7304	161.45	2.33	-12.48
Router (sft+grpo) exclu+exclu	-	-	-	-	0.8974	279.66	Dominant	+1.82
Router (sft+grpo) exclu+balance	-	-	-	-	0.8953	273.98	18.54	+3.12
Router (sft+grpo) exclu+origin	-	-	-	-	0.8932	273.74	8.75	+2.50

Table 4: Performance and Efficiency on DISC ($|\text{LLM}| = 6$) Dataset ($\alpha = 0.2$)

Key Takeaway: The strict 3:1 NBS perfectly filters out routers that previously "cheated" by indiscriminately dropping expensive models. Baselines like Random, Emb+MLP, and GRPO-RL now show deep negative NBS scores. In contrast, our proposed *sft+grpo* methods withstand the penalty, yielding the highest TER among non-oracle models and securing a solid positive NBS, proving high precision in cost-cutting.

Strategy	Standard (Normal)				Exclusive (exclu)			
	HR ($\uparrow$)	Cost ($\downarrow$)	TER ($\uparrow$)	NBS ($\uparrow$)	HR ($\uparrow$)	Cost ($\downarrow$)	TER ($\uparrow$)	NBS ($\uparrow$)
Strongest Baseline	0.8597	221.68	-	-	0.8597	221.68	-	-
Largest Baseline	0.8597	221.68	-	-	0.8597	221.68	-	-
Smallest Baseline	0.5052	44.87	1.93	-43.96	0.5052	44.87	1.93	-43.96
Router (oracle)	1.0000	111.27	Dominant	+98.77	1.0000	111.27	Dominant	+98.77
Router (random)	0.6803	153.80	1.47	-31.98	0.6789	153.13	1.47	-32.17
Router (knn)	0.8271	192.50	3.47	+1.79	0.7945	170.00	3.08	+0.56
Router (emb+MLP)	0.7802	175.58	2.25	-6.95	0.7743	163.27	2.65	-3.45
Router (emb+KNN+ELO)	0.8268	193.75	3.30	+1.12	0.7951	170.50	3.07	+0.55
Router (BERT_MLC)	0.7845	138.94	4.27	+11.08	0.7851	185.83	1.86	-9.86
Router (Qwen2.5-3B Base)	0.6470	139.28	1.75	-26.64	-	-	-	-
Router (sft-oracle)	-	-	-	-	0.7677	171.96	2.10	-9.68
Router (grpo-rl)	0.7469	144.51	2.65	-4.55	0.7250	168.40	1.78	-16.38
Router (sft+grpo) exclu+exclu	-	-	-	-	0.8512	210.03	5.32	+2.29
Router (sft+grpo) exclu+balance	-	-	-	-	0.8506	209.21	5.32	+2.45
Router (sft+grpo) exclu+origin	-	-	-	-	0.8392	198.87	5.02	+4.14

Deployment Recommendations Based on the experimental results across all datasets, we observe a clear behavioral distinction between routing strategies:

- **Precision & Quality-First (sft+grpo):** Our proposed **sft+grpo** methods consistently secure the highest Hit Rates among all non-oracle strategies, ensuring strong reliability. Because they are heavily optimized to protect accuracy, their primary limitation is that cost reduction is sometimes constrained by this "accuracy pull." If your production environment strictly demands high precision and minimal performance degradation, **sft+grpo** is undoubtedly the superior choice.
- **Aggressive Cost-Cutting (Traditional Routers):** In scenarios where absolute accuracy is less critical and the primary objective is to maximize budget savings, traditional methods like **KNN** or **Emb+MLP** may be more suitable. They tend to operate with wider margins, willing to trade away larger portions of accuracy in exchange for deeper cost reductions.

5.2 Training & Computation Time Efficiency

To provide a comprehensive view of the operational overhead, we recorded the total training and computation times for each routing strategy. All benchmarks were executed on a single NVIDIA GeForce RTX 3090 GPU, and the reported figures represent the average time across three independent runs.

Table 5: Training and Computation Time (in Seconds) Across Datasets

Key Takeaway: *Traditional embedding-based ML routers are exceptionally fast to fit, completing in seconds. The proposed **sft+grpo** architecture incurs a significant one-time training cost (~ 1.3 hours) due to the autoregressive generation required during RL rollouts. However, because GRPO training is strictly capped at 650 steps, this upper bound of computational time remains stable and predictable regardless of dataset size.*

Routing Strategy	HS_3LLM	HS_6LLM	MMLU_6LLM	Disc_6LLM
Router (knn)	13.9 s	13.9 s	13.4 s	20.5 s
Router (emb+MLP)	14.6 s	14.5 s	11.1 s	21.0 s
Router (emb+KNN+ELO)	12.8 s	12.1 s	10.2 s	24.0 s
Router (BERT_MLC)	1,341.3 s	1,119.5 s	252.6 s	1,342.5 s
Router (sft-oracle)	679.0 s	574.1 s	153.0 s	666.5 s
Router (sft+grpo)	4,651.0 s	4,554.7 s	4,682.1 s	4,721.9 s

Operational Insights:

- **Instantaneous Fitting for Traditional ML:** Models relying purely on static embeddings (KNN, Emb+MLP) finish computations in under 25 seconds. They simply map text to vectors and perform basic mathematical clustering or linear mapping, making them ideal for ultra-rapid prototyping.
- **Dataset Size Dependency (SFT & BERT-MLC):** The training time for cross-encoder architectures (BERT-MLC) and Phase-1 SFT strictly correlates with the dataset capacity (N). For instance, training SFT on the large HellaSwag dataset ($N = 3479$) takes roughly 679 seconds, but plummets to just 153 seconds on the severely truncated MMLU dataset ($N = 529$).

- **The GRPO Computational Ceiling:** The SFT+GRPO architecture is the most computationally expensive, averaging around 4,600 seconds (~ 1.3 hours). This is due to the inherent mechanics of Reinforcement Learning: the router must physically perform auto-regressive text generation (rollout) to explore the action space before evaluating the reward. Notably, because we purposefully constrain the GRPO phase to a maximum of 650 steps, the training time remains completely stable across all datasets. This one-time offline training cost acts as an acceptable and highly controllable trade-off for the massive "Dominant" precision it unlocks during online inference.

5.3 Inference Latency in Production (Batch Size = 1)

While training is a one-time offline cost, inference latency represents the real-time operational overhead added to every user query. In a production LLM Gateway, queries typically arrive sequentially rather than in perfectly batched parallel sets. To simulate this real-world sequential arrival, we evaluated the inference latency on 1,000 queries from the DISC dataset with a strict **Batch Size = 1**.

We compared the strongest embedding-based baseline (**BERT-MLC**) against our proposed autoregressive routing architecture (**SFT+GRPO**).

Table 6: Inference Latency and Throughput (DISC Dataset, $N = 1000$, Batch Size = 1)

Routing Architecture	Total Time (s)	Average Latency (ms / query)	Throughput (queries / sec)
Router (BERT_MLC)	13.24 s	13.24 ms	75.50 q/s
Router (sft+grpo)	197.81 s	197.81 ms	5.06 q/s

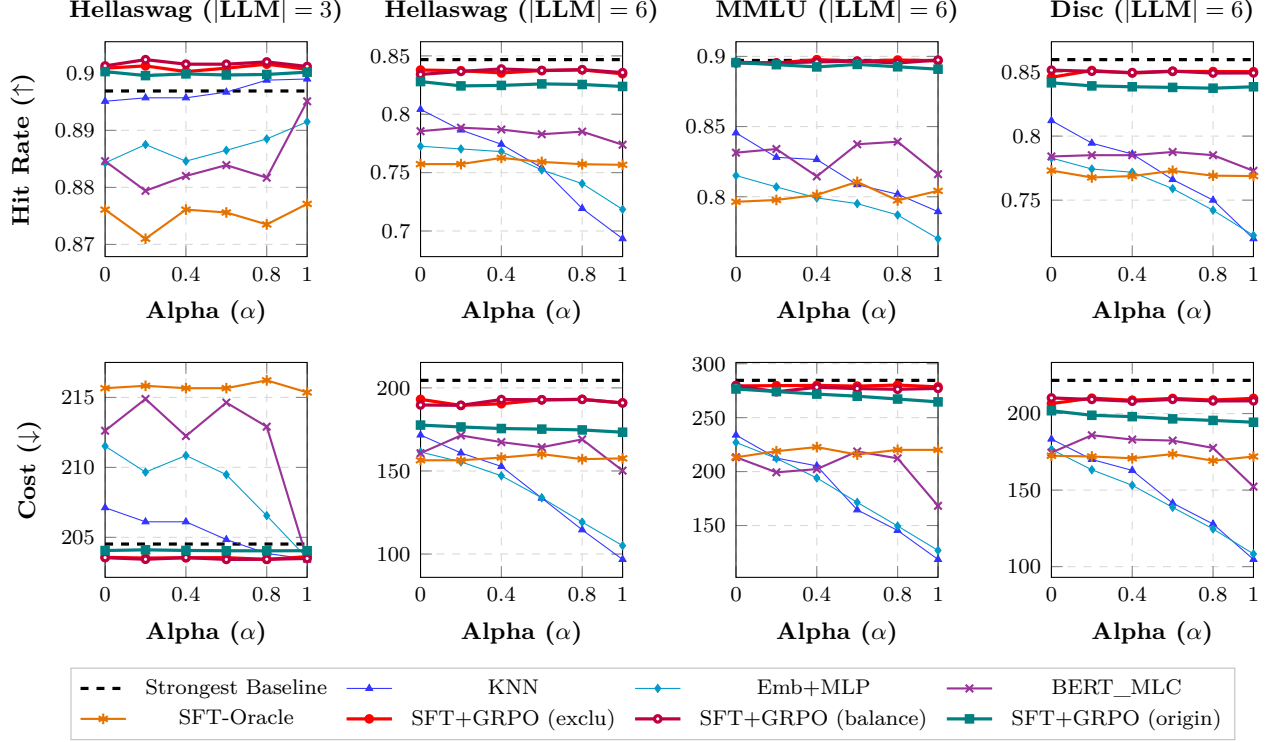
Architectural & Operational Analysis:

- **Encoder vs. Decoder Mechanics:** **BERT-MLC** achieves extreme speed because it relies on a single forward pass through a relatively small encoder network (~ 100 M parameters) combined with a classification head. In contrast, **SFT+GRPO** utilizes a 3-Billion parameter decoder-only model (Qwen-2.5-3B). Even though it only needs to generate a few tokens to output the routing decision, initializing the KV cache and executing the autoregressive forward passes inherently consumes more time.
- **The Production Perspective:** In strict isolation, a $15\times$ slowdown seems drastic. However, context is crucial. In an end-to-end LLM pipeline, generating a response from a large frontier model usually takes between 3 to 10 seconds. Adding a ~ 0.2 -second (197 ms) latency overhead at the Gateway level is virtually imperceptible to the end-user.

6 Cost-Aware Analysis

6.1 Alpha Penalty Spectrum

To comprehensively evaluate how different routing strategies handle increasing token-saving pressure, we plot their performance across six Alpha (α) values ranging from 0.0 (No Penalty) to 1.0 (Maximum Penalty).



Deep Analytical Insights

Analyzing the curves reveals clear architectural behaviors:

- 1. Upper Bounds of Accuracy (Exclu/Balance Variants):** For the SFT+GRPO (exclu and balance) variants, since the training data instances are strictly mapped to a single correct LLM candidate, the cost differential during the rollout phase is effectively nullified. These two variants act purely as accuracy-driven strategies, demonstrating the theoretical upper bound of the architecture’s accuracy.
- 2. Cost-Awareness in the Origin Variant:** The SFT+GRPO (origin) variant is trained on the original dataset where multiple models can answer a prompt correctly. This non-uniqueness allows the GRPO rollout phase to explore various correct options, seamlessly reflecting cost differences in the reward signal. Consequently, it learns true “cost-awareness,” achieving greater cost savings than the exclusive/balance variants, albeit with a slight trade-off in accuracy.
- 3. The Double-Edged Sword of Stability:** As observed across the charts, a core characteristic of the SFT+GRPO fine-tuning framework is its tendency to firmly anchor high accuracy while attempting to compress costs. Because of this strong accuracy priority, its cost does not

drop as drastically or as easily as other baselines. This remarkable stability is both its greatest advantage and a limitation under extreme token-saving demands.

- **4. The "Large Pool Bonus" Phenomenon ($|\mathbf{LLM}| = 6$):** When the candidate LLM pool expands, routing complexity naturally increases. This causes the model's routing decision signals to shift slightly from pure accuracy certainty toward cost-awareness. In scenarios where the router struggles to confidently identify the absolute best model, it tends to default to cheaper alternatives. Consequently, costs generally shift downward across all strategies—a natural "bonus" of enlarging the LLM pool.
- **5. The Hellaswag "Cost-Accuracy Paradox" (The Anomaly in $|\mathbf{LLM}| = 3$):** An anomaly occurs in the original Hellaswag dataset ($|\mathbf{LLM}| = 3$): as α increases, cost drops, yet the Hit Rate slightly *increases*. Here, the most expensive model lacks a distinct advantage. As heavy token penalties drive the reward for this mediocre model to zero, it is pruned from the action space. This eliminates routing confusion, allowing the router to focus purely on the two highly efficient experts—yielding both lower costs and higher accuracy.

6.2 Routing Distribution Evolution: SFT to Cost-Aware GRPO

To deeply understand how the reward function physically alters the router’s decision-making policy, we visualize the routing distribution (Predicted Destination vs. Ground Truth) at three distinct training stages. This visual evolution explicitly proves that our architecture successfully transitions from pure static feature-matching into dynamic, cost-aware reward maximization.

To ensure clarity and legibility, the following heatmaps are presented individually.

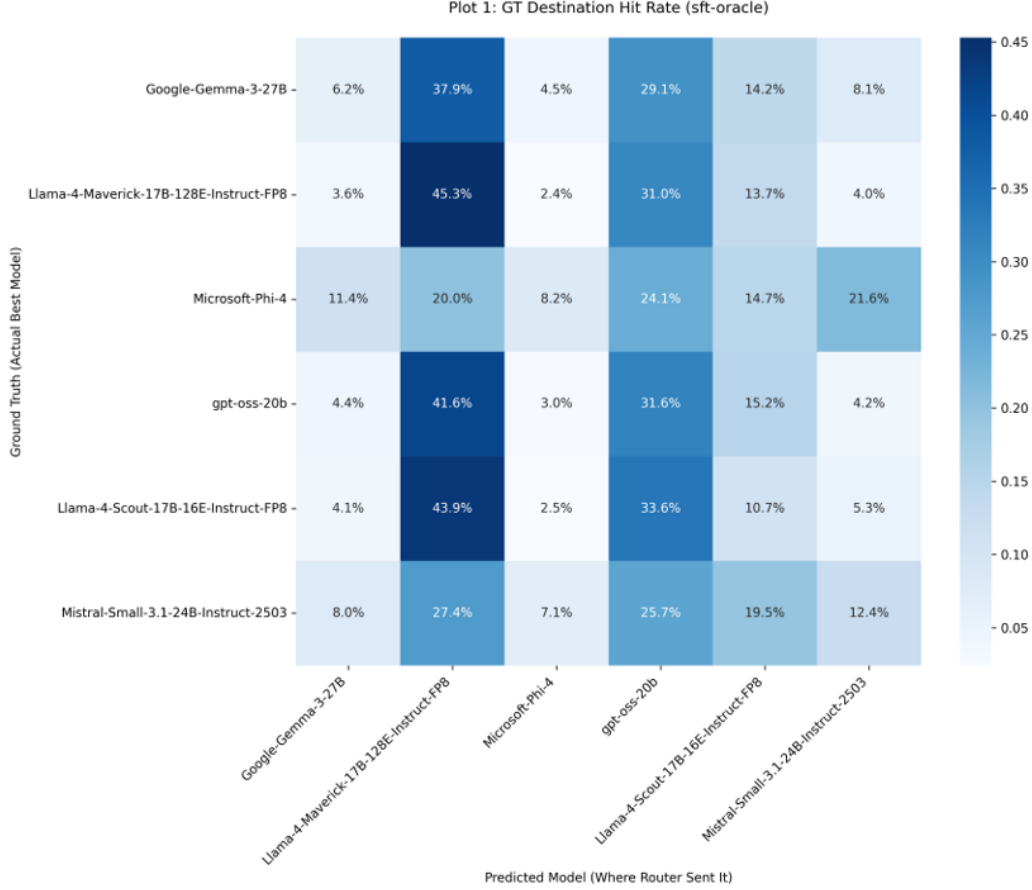


Figure 2: **Stage 1: SFT-Oracle Baseline.** At this stage, the routing distribution strictly mimics the optimal Oracle labels. The model acts purely on semantic feature extraction, focusing on learning the rigid boundaries between different task domains. This results in a highly distributed routing pattern with distinct capability boundaries.

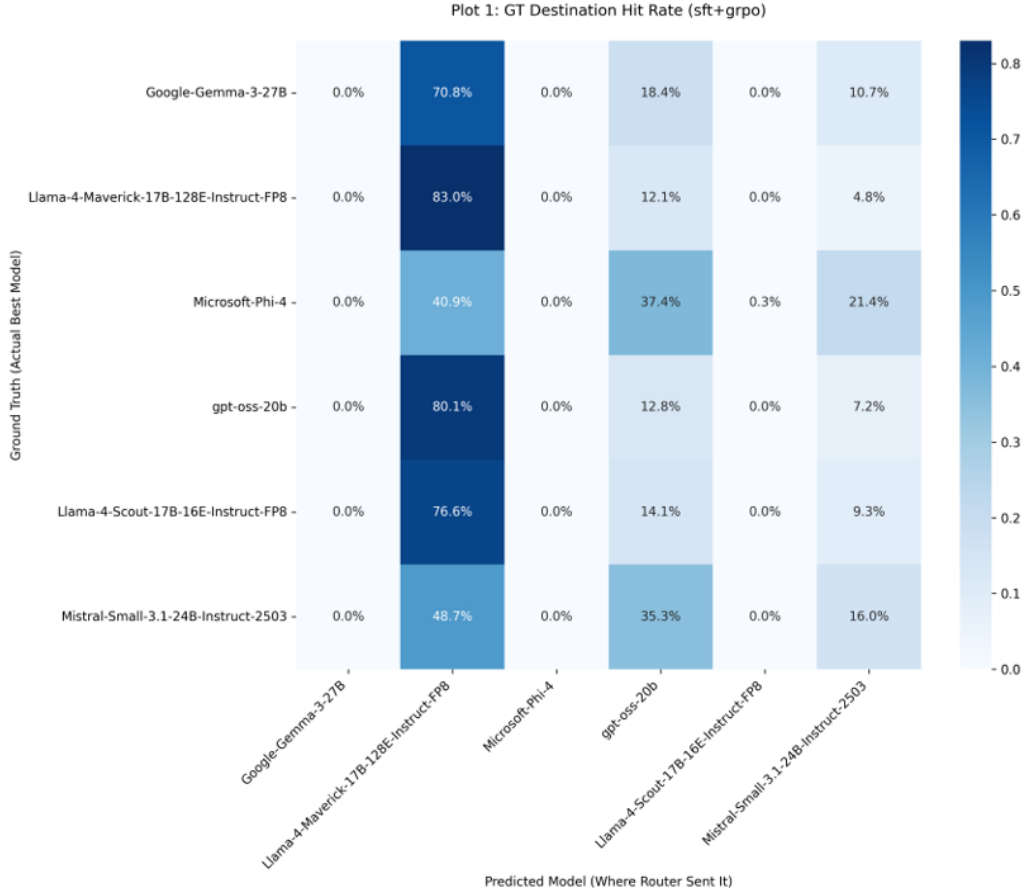


Figure 3: **Stage 2: GRPO ($\alpha = 0$, Pure Accuracy-Driven)**. When the model transitions to the Reinforcement Learning stage utilizing the original dataset (without any cost penalty), the policy fundamentally shifts. Driven purely by the correctness reward, the router abandons strict SFT feature-matching and aggressively converges toward the models that statistically guarantee the highest Hit Rates. This confirms that the RL reward signal effectively overrides the static SFT priors.

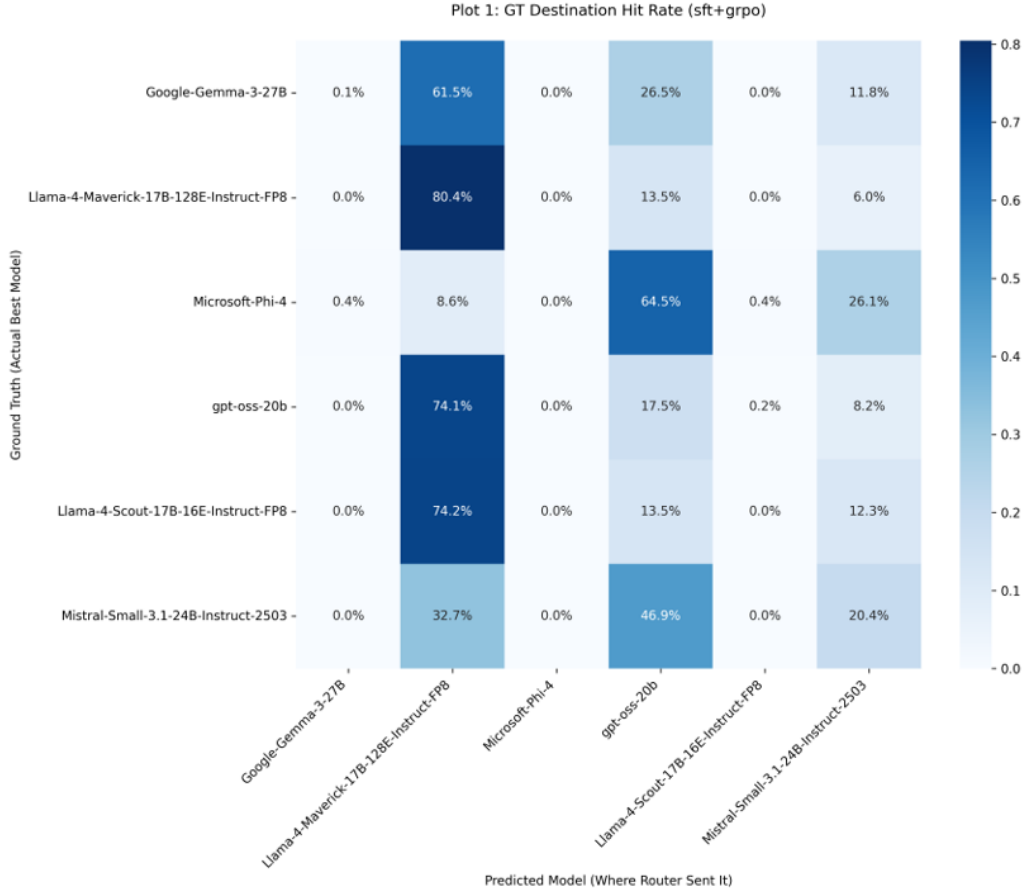


Figure 4: **Stage 2: GRPO** ($\alpha = 1.0$, **Maximum Cost-Awareness**). Under extreme cost pressure, the router exhibits explicit budget-awareness. We observe a structural migration of inference traffic: queries originally routed to expensive frontier models (e.g., Llama-4-Maverick) are aggressively diverted to highly cost-effective alternatives, most notably `gpt-oss-20b` (normalized cost ~ 0.302). This serves as empirical proof that our proposed Conditional Cost Penalty mathematically enforces and achieves dynamic budget control.

7 Root-Cause Analysis of Dataset Routing Value

To systematically understand why the routing architecture converges efficiently on certain datasets (e.g., HellaSwag) but struggles on others (e.g., MMLU), we propose a **Four-Dimensional Root-Cause Framework**. This framework divides the dataset quality into two main axes: the *Data-Side* (input feature distribution) and the *GT-Side* (Ground-Truth label dynamics).

7.1 The Four-Dimensional Analytical Framework

The framework relies on four mathematical metrics to quantify the "learnability" and "routing signal strength" of a dataset. Removing redundant overlap, these metrics directly pinpoint the physical capacity, semantic coherence, discriminative variance, and label-feature alignment.

Table 7: Mathematical Definitions and Interpretations of the Four Metrics

Metric	Mathematical Formula	Intuitive Meaning (Core Significance)
1. Samples (N)	$N = D_{\text{train}} $	Statistical Baseline (Data-Side): The physical capacity for generalization. Higher means the router observes more variations to learn stable patterns; Lower increases the risk of over-fitting.
2. Avg_Sim	$\text{Sim} = \frac{1}{N(N-1)} \sum_{i \neq j} \cos(\mathbf{v}_i, \mathbf{v}_j)$	Semantic Coherence (Data-Side): Measures how tightly clustered the queries are in the embedding space. Higher similarity enables stable, unified feature extraction. Lower means queries are scattered randomly.
3. Dec_Var (σ^2)	$\sigma^2 = \frac{1}{M} \sum_{k=1}^M (\text{Acc}_k - \overline{\text{Acc}})^2$	Discriminative Gain (GT-Side): The variance in accuracy among candidate models. Higher variance means the dataset effectively separates experts from novices. Lower means all models perform similarly, resulting in flat, ambiguous reward signals.
4. CH Score	$CH = \frac{SS_B}{SS_W} \times \frac{N-K}{K-1}$	Cluster Separation (GT-Side): Measures how well text features group according to their GT labels [3]. Higher means different expert domains are distinctly separated and internally cohesive. Lower means domain boundaries overlap chaotically.

Detailed Formula Breakdown & Symbol Definitions

To ensure mathematical transparency and facilitate reproducibility, we provide a detailed expansion of the core formulas and define each operational symbol used within the Four-Dimensional Framework:

- **Semantic Coherence (Avg_Sim):**

$$\text{Sim} = \frac{1}{N(N-1)} \sum_{i \neq j} \cos(\mathbf{v}_i, \mathbf{v}_j)$$

Where N is the total number of text samples (queries) in the dataset, and $\mathbf{v}_i$ and $\mathbf{v}_j$ represent the high-dimensional embedding vectors of the i -th and j -th queries, respectively. The function $\cos(\cdot, \cdot)$ computes the cosine similarity between the two vectors. This metric captures the global average pairwise semantic proximity of the dataset.

- **Discriminative Gain (Dec_Var, σ^2):**

$$\sigma^2 = \frac{1}{M} \sum_{k=1}^M (Acc_k - \overline{Acc})^2$$

Where M defines the total number of candidate LLMs available in the routing pool. Acc_k denotes the baseline accuracy (or win rate) of the k -th candidate model on the given dataset, while $\overline{Acc}$ is the arithmetic mean of the accuracies across all M models.

- **Cluster Separation (CH Score - Calinski-Harabasz Index):**

$$CH = \frac{SS_B}{SS_W} \times \frac{N - K}{K - 1}$$

The CH Score fundamentally evaluates whether the Ground-Truth (GT) labels form valid, mathematically separable clusters in the semantic space. Here, N is the total number of queries, and K represents the total number of unique Ground-Truth label combinations (i.e., the distinct expert routing domains).

1. Between-cluster Variance (SS_B):

$$SS_B = \sum_{k=1}^K n_k \|C_k - C_{\text{global}}\|^2$$

Where n_k is the number of samples strictly assigned to the k -th expert cluster, C_k is the geometric centroid (mean vector) of that specific cluster, and C_{global} is the overall centroid of the entire dataset. A larger SS_B indicates that the semantic territories of different LLM experts are sharply separated (e.g., math queries vs. conversational queries are topologically distant).

2. Within-cluster Variance (SS_W):

$$SS_W = \sum_{k=1}^K \sum_{i=1}^{n_k} \|X_{i,k} - C_k\|^2$$

Where $X_{i,k}$ corresponds to the i -th sample vector within the k -th expert cluster. This measures internal cohesion; a smaller SS_W demonstrates that queries intended for the same model are highly uniform in phrasing, structure, and vocabulary.

3. Normalization Factor ($\frac{N-K}{K-1}$): This fraction acts as a mathematical penalty to prevent score inflation caused by a large number of categories. By adjusting for Degrees of Freedom, it ensures that the CH Scores remain objectively comparable across datasets of completely different scales and model pool sizes.

7.2 Empirical Base Data: LLM Win Rates

Before diving into the root-cause metrics, it is crucial to observe the raw win-rate distribution of the candidate LLMs across the three exclusive datasets. These distributions form the foundation of our GT-Side analysis.

Table 8: Exclusive Win Rates and Mathematical Variance Across Datasets

Candidate LLM	HS_3LLM ($N = 3,479$, 3 Models)	HS_6LLM ($N = 2,801$, 6 Models)	MMLU_6LLM ($N = 529$, 6 Models)	Disc_6LLM ($N = 3,321$, 6 Models)
Llama-4-Maverick-17B	67.35%	44.91%	31.19%	41.43%
gpt-oss-20b	–	17.53%	22.87%	27.85%
Llama-4-Scout-17B	19.29%	11.00%	15.69%	12.74%
Mistral-Small-3.1	13.37%	11.14%	11.34%	8.10%
Google-Gemma-3-27B	–	13.57%	9.64%	6.35%
Microsoft-Phi-4	–	1.86%	9.26%	3.52%
<i>Resulting Dec_Var (σ^2)</i>	0.0584	0.0219	0.0064	0.0184

Metric Connection (σ^2): The table above is a direct visual proof of the Dec_Var metric. In HS_3LLM, the win rates are heavily skewed (Maverick dominates at 67%), creating a steep capability gradient. In MMLU_6LLM, the distribution is “flattened,” with all 6 models compressed tightly between 9% and 31%. This mathematical separation defines how clear the reward signals will be for the Router.

7.3 Two-Sided Origin Analysis: Where Does the Signal Come From

We analyze the empirical differences between the HellaSwag (HS) and MMLU exclusive datasets by grouping the metrics into *Data-Side* (What the queries look like) and *GT-Side* (How the labels map onto the data and models).

Table 9: Empirical Metric Comparison Across Datasets

Dataset (Exclusive)	Samples (N)	Sim_Emb (Coherence)	Dec_Var (Variance)	CH Score (Separation)
HS_3LLM (3,479 queries)	3,479	0.0355	0.0584	5.6391
HS_6LLM (2,801 queries)	2,801	0.0246	0.0219	2.5305
Disc_6LLM (3,321 queries)	3,321	0.0302	0.0184	2.4207
MMLU_6LLM (529 queries)	529	0.0144	0.0064	1.0443

7.3.1 The Data-Side (Feature Representation)

Metrics involved: Samples (N), Avg_Sim

This side dictates whether the router can extract stable and generalized patterns from the text.

- **Sample Size Baseline (N):** For optimal training of the SFT+GRPO framework, a dataset size of approximately 3,000 samples serves as an ideal and sufficient volume to reach logical convergence. MMLU’s severely low sample count ($N = 529$) significantly impairs the router’s ability to generalize.
- **Semantic Coherence (Avg_Sim):** A higher semantic similarity implies that the data points possess structured features. Datasets like HellaSwag and DISC exhibit relatively cohesive semantic spaces, facilitating the router’s ability to map underlying textual patterns to specific routing destinations. Conversely, MMLU’s low coherence (0.0144) reflects a chaotic, fragmented feature space.

7.3.2 The GT-Side (Reward Landscape & Label Alignment)

Metrics involved: Dec_Var (σ^2), CH Score

This side dictates the strength of the reinforcement learning (GRPO) reward signal and whether the Ground-Truth labels actually form meaningful boundaries.

- **Model Strength Imbalance (σ^2):** An excessive gap in candidate model strength forces the router to heavily prefer the dominant LLM. Even when multiple models qualify as experts, the router will gravitate toward the absolute strongest one. To foster effective expert division, a smaller capability variance—where model win rates are relatively balanced—is highly recommended.
- **Label-Feature Alignment (CH Score):** This is the most critical metric for routing feasibility. The Calinski-Harabasz Index validates whether the GT labels actually correspond to distinct textual characteristics.
 - **Between-cluster Variance (SS_B):** A higher SS_B means that queries belonging to different LLM experts (e.g., math queries vs. common-sense queries) are located far apart in the semantic space.
 - **Within-cluster Variance (SS_W):** A lower SS_W means that queries routed to the same LLM are highly consistent in tone, length, and vocabulary.

The **HS_3LLM** dataset achieves a stellar CH Score of 5.6391, proving that its labels represent clear, well-separated expert domains. In stark contrast, **MMLU_6LLM** plummets to 1.0443. This mathematically proves that MMLU’s labels are misaligned with the text features—different expert queries overlap chaotically, completely destroying the router’s decision boundary.

7.4 Visualizing the Dynamics: Embedding Spaces and Routing Heatmaps

To practically validate the Four-Dimensional Framework, we visualize the feature spaces alongside their corresponding routing behaviors. **It is important to note that the routing behaviors (sft+grpo) analyzed in this section are specifically trained and evaluated on the Mutually Exclusive Set (exclu).** This ensures that our visual analysis of expert division is based on strict, single-correct-answer decision boundaries.

To ensure a comprehensive baseline for text feature representation, we utilize two distinct paradigms: (1) **BM25** [16], a traditional lexical retrieval algorithm based on exact keyword matching representing sparse, surface-level features; and (2) **Embedding Proxy** (True Embedding), which utilizes deep neural networks to project text into a continuous vector space. To maintain experimental consistency, the Embedding Proxy utilizes the exact same model employed by our ML Baselines (sentence-transformers/all-MiniLM-L6-v2).

Scenario 1: HellaSwag (High Separation, High Efficiency)

Figure 5 displays the t-SNE reduction [19] of HellaSwag queries clustered via K-means. Reflecting its high CH Score (5.6391), distinct topological boundaries emerge cleanly between different domain experts in the Embedding Proxy space. Even within the keyword-bound BM25 space, a recognizable core-periphery structure persists.

This robust data-to-label alignment directly empowers the router to establish precise decision boundaries. As a result, the destination hit-rate heatmap (Figure 6) showcases a highly successful execution of **Expert Routing**. Instead of yielding to a lazy, centralized assignment, the router adapts to the strong variance signals, accurately routing 51.3% of tasks to Mistral-Small and 79.2% to Microsoft-Phi-4 when they are the designated ground-truth specialists.

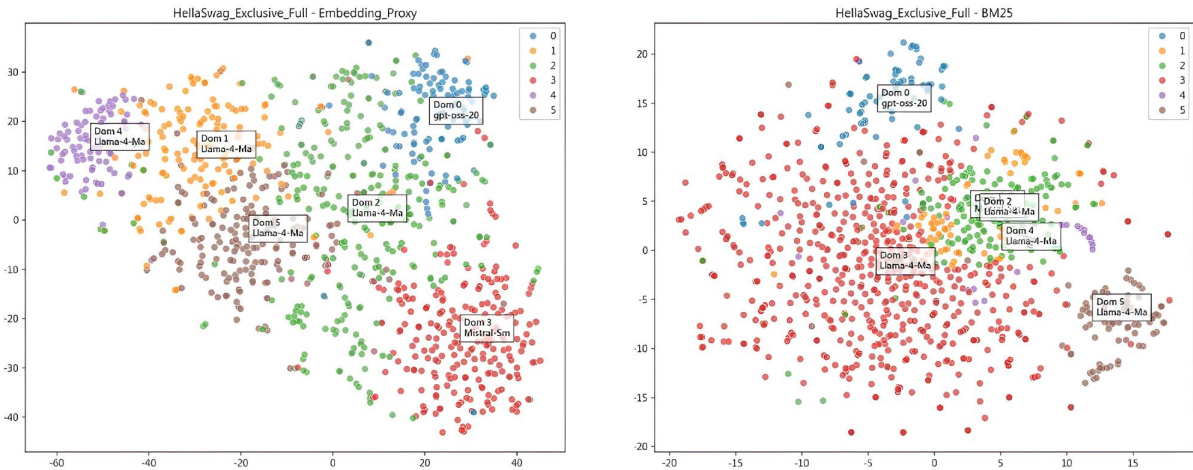


Figure 5: HellaSwag Feature Distributions: Clear clustering and distinct expert boundaries (reflecting high CH Score) visible under Embedding Proxy (left) and BM25 (right).

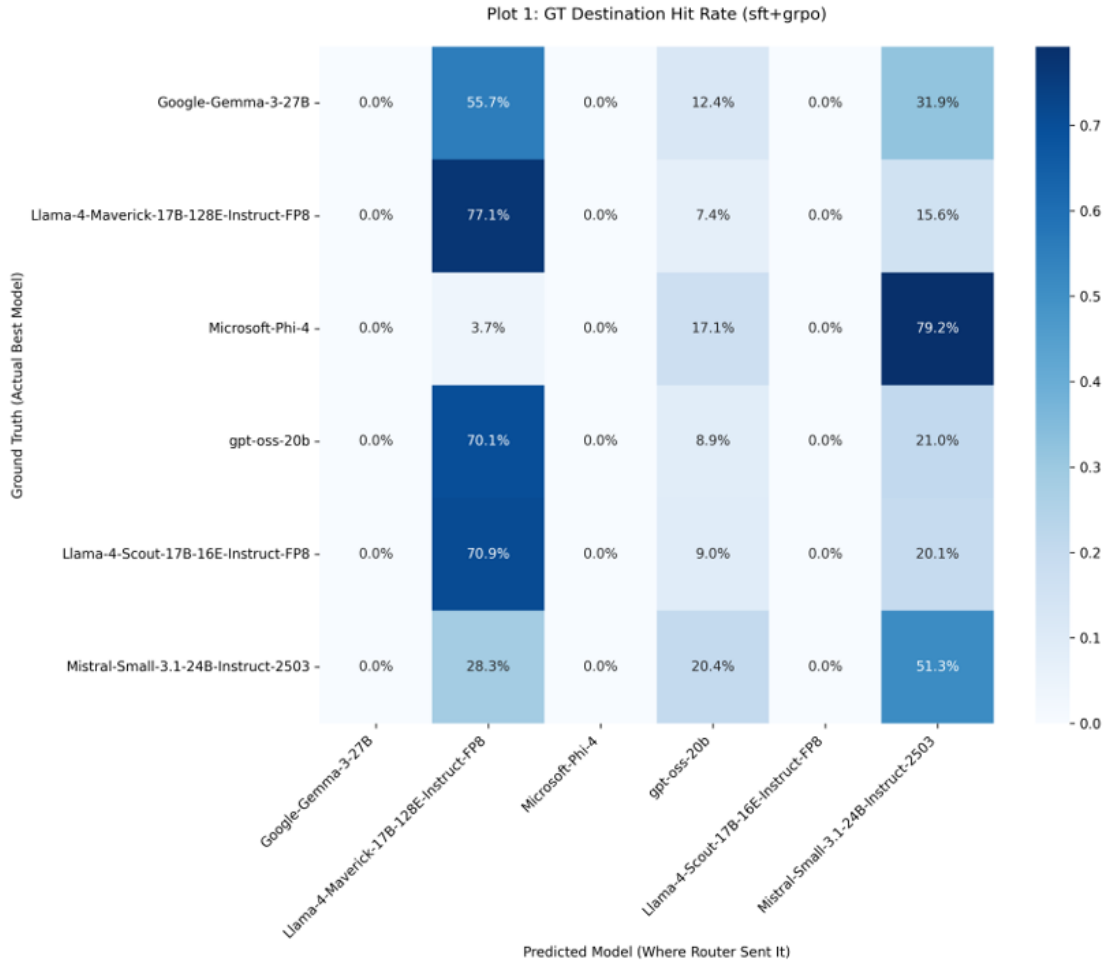


Figure 6: HellaSwag Routing Heatmap: Clear evidence of successful expert routing and adaptive workload distribution across specialized LLMs.

Scenario 2: MMLU (Low Separation, Routing Collapse)

In stark contrast, Figure 7 visually exposes the empirical pathology of MMLU, perfectly corroborating its abysmal CH Score (1.0443). The queries compress into an inseparable, chaotic "hairball." The domain labels overlap completely without any discernible topological boundaries, proving that SS_B is critically low while SS_W is massive.

When this blind feature space is paired with a flat, convergent reward landscape ($\sigma^2 = 0.0064$), the router faces an optimization deadlock. To avoid penalties during the GRPO phase, the router stops exploring classification paths and defaults to a risk-averse forwarding strategy.

As demonstrated in Figure 8, this culminates in a catastrophic **Routing Collapse** (Degeneration). The router completely ceases expert classification, blindly routing 93% to 100% of all incoming traffic across all ground-truth categories to a single strongest baseline (Maverick). The Mixture-of-Experts architecture effectively degenerates into a single-model inference pipeline.

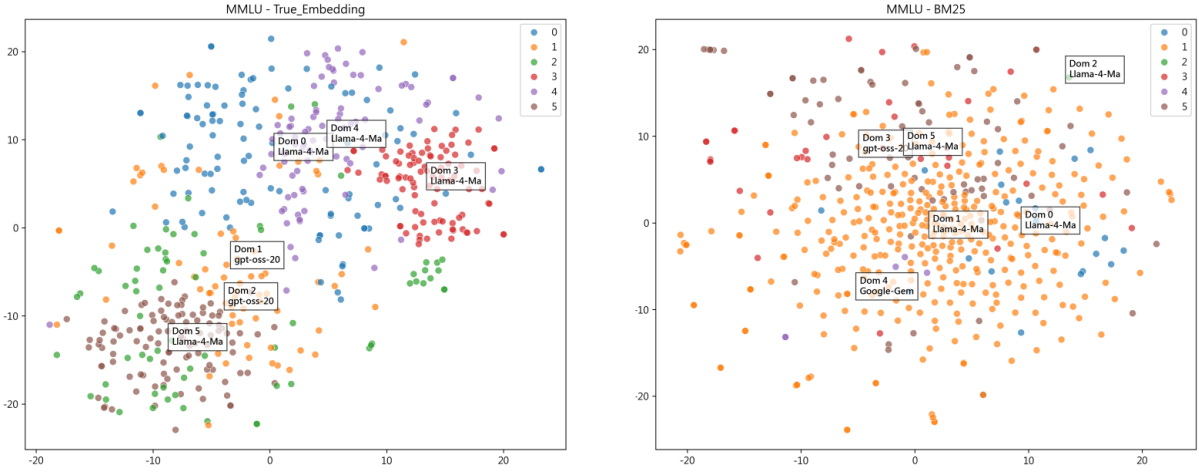


Figure 7: MMLU Feature Distributions: The severe lack of cluster separation visualized as an inseparable, overlapping "hairball."

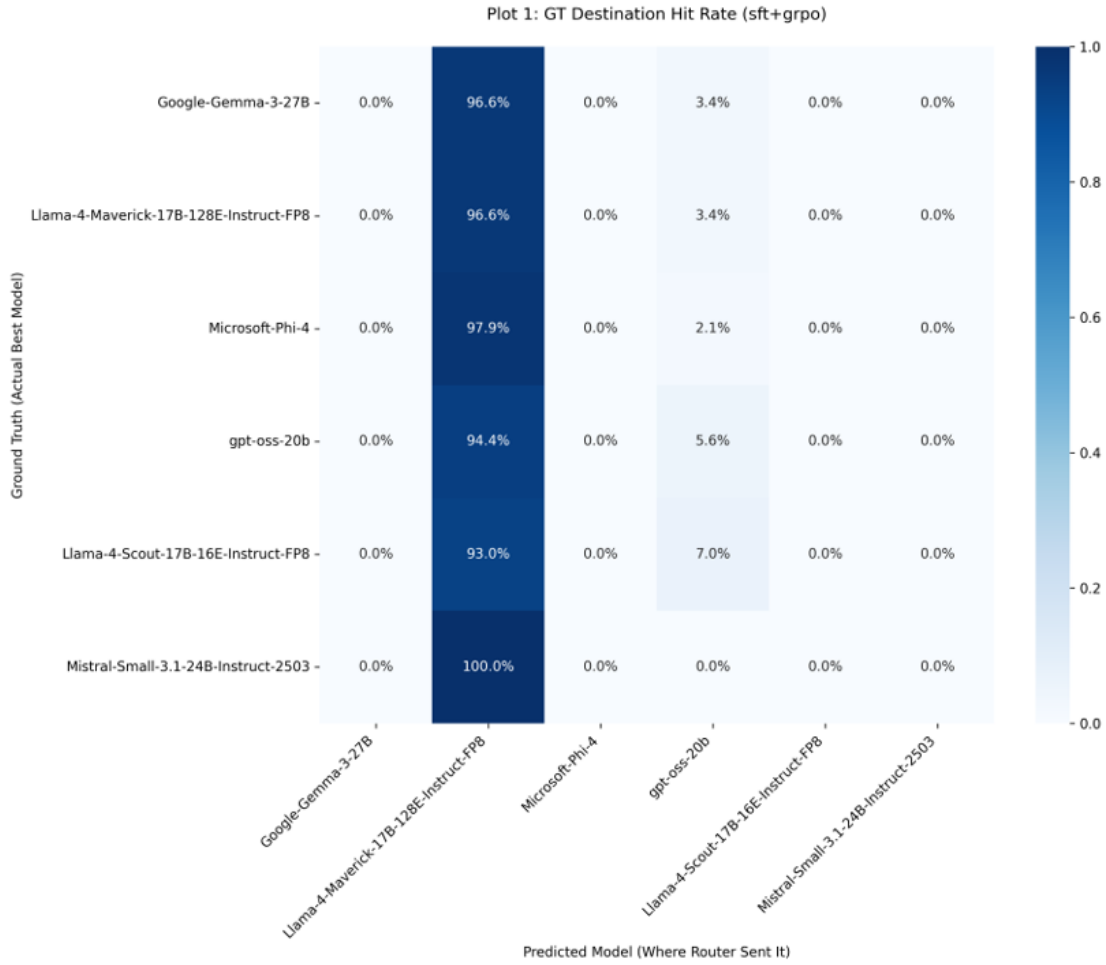


Figure 8: MMLU Routing Heatmap: A complete routing collapse where the router ceases expert classification and defaults to the strongest model.

8 Conclusion

To bridge the gap between mathematical metrics and practical engineering, it is crucial to understand what makes a dataset inherently "routable."

The Intuitive Feel of Good vs. Bad Routing Data

Intuitively, a **Highly Routable (Good) Dataset** feels like sorting distinctly different objects—such as assigning strict mathematical equations to a heavy reasoning model, and casual chitchat to a lightweight open-source model. The queries possess obvious stylistic, structural, or semantic "triggers" that perfectly align with a specific model's inherent strength.

Conversely, a **Poor (Bad) Dataset** feels like sorting indistinguishable grey boxes. For example, two highly complex medical queries might look structurally identical, yet Model A randomly succeeds on the first while Model B succeeds on the second. If a human expert cannot glance at a prompt's structure and intuitively guess which model tier is best suited for it, the routing algorithm will similarly fail to find a mathematical boundary. It will eventually collapse into random guessing or default entirely to the strongest, most expensive baseline.

Engineering Priorities & Recommended Thresholds

When engineering or filtering a dataset to train an expert router via the SFT+GRPO framework, we strongly recommend prioritizing the metrics from our Four-Dimensional Framework in the following order:

- **Priority 1: CH Score (Label-Feature Alignment)**
 - **Why it matters most:** This is the absolute prerequisite for learnability. It dictates whether your desired routing destinations (labels) actually correspond to structurally distinct textual patterns (features).
 - **Recommended Threshold:** $CH > 2.0$
 - **Actionable Advice:** If your dataset scores below 2.0 (e.g., MMLU at 1.04), the training will almost certainly collapse. You must either re-embed the text using a more sensitive embedding model or aggressively merge Ground-Truth categories to force higher separation.
- **Priority 2: Avg_Sim (Semantic Coherence)**
 - **Why it matters:** Before the router can map features to labels, the features themselves must be stable and recognizable. If the dataset is too sparse or chaotic, the model cannot extract reliable routing triggers, regardless of how good the reward signal is.
 - **Recommended Threshold:** $Avg_Sim > 0.025$
 - **Actionable Advice:** If the score falls below 0.025 (e.g., MMLU at 0.014), the dataset represents an overlapping "hairball" of disconnected topics. You should narrow the domain scope of the dataset or increase the sample density within specific task categories to build coherent clusters.
- **Priority 3: Dec_Var (σ^2 , Discriminative Gain)**
 - **Why it matters:** Once learnability is confirmed (Priorities 1 & 2), this metric determines the ROI (Return on Investment) of routing. If the variance is too low, it means all your candidate models are equally good (or equally bad) at the tasks.

- **Actionable Advice:** If σ^2 is below 0.015, the reward signals during the GRPO phase will be too flat to guide the policy efficiently. In such scenarios, dynamic routing is mathematically unnecessary; you should simply prune redundant models from the pool and deploy the single cheapest model that meets your baseline accuracy requirements.

The proposed Two-Stage Cost-Aware Routing Architecture excels on datasets like HellaSwag and DISC because they offer distinct structural boundaries ($SS_B \gg SS_W$) and coherent semantic spaces, allowing the router to physically map text representations to specific model capabilities.

Conversely, MMLU fails to train effective routers not simply because the questions are "hard," but because its landscape is structurally chaotic (low Avg_Sim) and its labels completely lack semantic separation (low CH Score). To achieve successful Mixture-of-Experts routing in production environments, data engineering must fiercely optimize for **feature-to-label alignment** (CH Score), ensure a **coherent feature space** (Avg_Sim), and maintain **sufficient capability gaps** among the candidate models (σ^2). Only when these data-side conditions are strictly met can the GRPO framework effectively push the router to its theoretical maximum of cost-efficiency.

References

- [1] Marah Abdin, Jyoti Aneja, Harkirat Behl, Sébastien Bubeck, Ronen Eldan, Suriya Gunasekar, Michael Harrison, Russell J Hewett, Mojan Javaheripi, Piero Kauffmann, et al. Phi-4 technical report. *arXiv preprint arXiv:2412.08905*, 2024.
- [2] Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K Arora, Yu Bai, Bowen Baker, Haiming Bao, et al. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925*, 2025.
- [3] Tadeusz Caliński and Jerzy Harabasz. A dendrite method for cluster analysis. *Communications in Statistics-theory and Methods*, 3(1):1–27, 1974.
- [4] Thomas Cover and Peter Hart. Nearest neighbor pattern classification. *IEEE transactions on information theory*, 13(1):21–27, 1967.
- [5] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient fine-tuning of quantized llms. *Advances in neural information processing systems*, 36:10088–10115, 2023.
- [6] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020.
- [7] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Liang Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. *Iclr*, 1(2):3, 2022.
- [8] Yehuda Koren, Robert Bell, and Chris Volinsky. Matrix factorization techniques for recommender systems. *Computer*, 42(8):30–37, 2009.
- [9] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.

- [10] Meta AI. Llama-4-Maverick-17B-128E-Instruct Model Card. <https://huggingface.co/meta-llama/Llama-4-Maverick-17B-128E-Instruct>, 2026. Accessed: 2026-05-30.
- [11] Meta AI. Llama-4-Scout-17B-16E Model Card. <https://huggingface.co/meta-llama/Llama-4-Scout-17B-16E>, 2026. Accessed: 2026-05-30.
- [12] Mistral AI. Mistral-Small-3.1-24B-Instruct-2503 Model Card. <https://huggingface.co/mistralai/Mistral-Small-3.1-24B-Instruct-2503>, 2025. Accessed: 2026-05-30.
- [13] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E Gonzalez, M Waleed Kadous, and Ion Stoica. Routellm: Learning to route llms with preference data. *arXiv preprint arXiv:2406.18665*, 2024.
- [14] Shishir G Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E Gonzalez. The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In *Forty-second International Conference on Machine Learning*, 2025.
- [15] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*, pages 3982–3992, 2019.
- [16] Stephen E Robertson, Steve Walker, Susan Jones, Micheline M Hancock-Beaulieu, Mike Gatford, et al. Okapi at trec-3. In *Overview of the Third Text REtrieval Conference (TREC-3)*, pages 109–126, 1994.
- [17] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- [18] Gemma Team. Gemma 3, 2025.
- [19] Laurens Van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *Journal of machine learning research*, 9(11), 2008.
- [20] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. Minilm: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. *Advances in neural information processing systems*, 33:5776–5788, 2020.
- [21] Benjamin Warner, Antoine Chaffin, Benjamin Clavié, Orion Weller, Oskar Hallström, Said Taghadouini, Alexis Gallagher, Raja Biswas, Faisal Ladhak, Tom Aarsen, et al. Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2526–2547, 2025.
- [22] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Wang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Ma, Jianxin Yang, Jin Xu, Jingren Zhou, Jinze Bai, Jinzheng He, Junyang Lin, Kai Dang, Keming Lu, Keqin Chen, Kexin Yang, Mei Li, Mingfeng Xue, Na Ni, Pei Zhang, Peng Wang, Ru Peng, Rui Men, Ruize Gao, Runji Lin, Shijie Wang, Shuai Bai, Sinan Tan, Tianhang Zhu, Tianhao Li, Tianyu Liu, Wenbin Ge, Xiaodong Deng, Xiaohuan Zhou, Xingzhang Ren, Xinyu Zhang, Xipin Wei, Xuancheng Ren,

Xuejing Liu, Yang Fan, Yang Yao, Yichang Zhang, Yu Wan, Yunfei Chu, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, Zhifang Guo, and Zhihao Fan. Qwen2 technical report, 2024.

- [23] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? In *Proceedings of the 57th annual meeting of the association for computational linguistics*, pages 4791–4800, 2019.
- [24] Haozhen Zhang, Tao Feng, and Jiaxuan You. Router-r1: Teaching llms multi-round routing and aggregation via reinforcement learning, 2025.

A Appendix

A.1 Prompt Architecture

During the GRPO rollout phase, the generation logic is guided by a specifically structured system prompt. To stabilize the reinforcement learning process and avoid sparse rewards caused by formatting errors, the prompt enforces a multi-step reasoning protocol (Analyze → Evaluate → Decide) before enforcing a strict output syntax.

A.2 System Prompt Template

You are an intelligent router aimed at optimizing cost-efficiency.
Your goal is to route the user's input to the cheapest model that can answer it correctly.

{menu_str}

Instructions:

1. "Analyze": First, analyze the complexity and specific requirements of the user's question.
2. "Evaluate": Compare the capabilities of the available models against these requirements.
3. "Decide": Select the most cost-efficient model capable of handling the task.
4. "Format": Ends your response strictly with the format: '[[Index]]'.

Example Response:

The user asks for a simple factual lookup. Model x is the cheapest and capable enough.
Model y is overkill.

Final Decision: [[x]]